

RELATÓRIO TÉCNICO ALTERNATIVO

PROJETO VR NO METAVERSO

Web 3.0 | Residência em TIC 29 — Atividade Avaliativa — Fase 2

SentinelVR: Central de Monitoramento Inteligente com Detecção de Anomalias

Idarlan Rogério Dias Magalhães | Turma 7

SEÇÃO 1 — IDENTIFICAÇÃO

Campo	Informação
Nome Completo	Idarlan Rogério Dias Magalhães
Turma / Residência	Turma 7 — Residência em TIC 29
Limitação de Hardware	Placa de vídeo integrada Intel UHD Graphics sem suporte ao Vulkan API exigido pelo Unity Editor com Meta XR SDK ativo. A GPU integrada não suporta renderização estereoscópica em tempo real com taxa de quadros mínima de 72 FPS exigida pelo Meta Quest, inviabilizando testes práticos do ambiente VR diretamente na máquina disponível.

SEÇÃO 2 — CONCEITO DO PROJETO

2.1 Nome do Projeto

SentinelVR: Central de Monitoramento Inteligente com Detecção de Anomalias

2.2 Contexto e Objetivo no Metaverso

O projeto SentinelVR propõe uma central de segurança em Realidade Virtual onde a própria imersão em VR é o núcleo funcional da solução — não apenas sua interface. O problema central é que operadores humanos de segurança sofrem de fadiga atencional ao monitorar múltiplas câmeras em telas 2D planas: o campo de visão é estreito, a varredura visual é forçada e mecânica, e eventos críticos em câmeras periféricas são frequentemente ignorados.

A VR resolve esse problema de forma estrutural, e não cosmética, por três razões intrinsecamente ligadas às propriedades do meio:

- **Espacialidade 360°:** Os 6 feeds de câmera são dispostos ao redor do operador em semicírculo imersivo. O operador não varre monitores — ele gira o corpo e o olhar naturalmente, criando memória espacial de onde cada câmera está. Isso é impossível em 2D.
- **Áudio espacial direcional:** Quando a IA detecta anomalia, o alerta sonoro é emitido na direção 3D da câmera afetada (spatialBlend = 1.0 com OVRSpatialAudio). O operador literalmente ouve de onde vem a ameaça antes de olhar para ela — reduzindo o tempo de reação.

- **Presença e engajamento ativo:** A sensação de presença física dentro da sala de controle mantém o operador cognitivamente ativo durante longos turnos. Dispensar alertas via gesto físico (grip do controlador) cria memória motora que reforça o treinamento para situações reais.

Um modelo de IA baseado em Autoencoder Convolucional analisa continuamente os frames das câmeras via pipeline Python e emite alertas visuais e sonoros automaticamente, liberando o operador para reagir apenas a eventos reais — não para monitoramento constante.

2.3 Descrição Geral do Ambiente Virtual

Uma sala de controle de segurança futurista de 12×10 metros com estética sci-fi. Paredes revestidas de painéis metálicos escuros com detalhes em LED azul elétrico. Iluminação ambiente baixa (atmosfera de trabalho noturno), com point lights sobre cada estação.

No centro, uma mesa hexagonal de vidro translúcido sobre a qual flutuam 6 monitores virtuais (Canvas World Space) em semicírculo de 180°, cada um exibindo o feed ao vivo de uma câmera de vigilância via RenderTexture. O semicírculo envolve fisicamente o operador — quando ele está na posição central, todos os monitores estão dentro do seu campo de visão periférico natural, algo impossível em uma sala física com telas 2D.

Dois painéis de alerta maiores flanqueiam o semicírculo, piscando em vermelho quando a IA detecta anomalia. O áudio espacial (OVRSpatialAudio) posiciona o som de alarme no espaço 3D da câmera afetada. Na periferia: prateleiras com equipamentos decorativos, painel de status do modelo de IA e partículas holográficas azuis reforçando a atmosfera tecnológica. Skybox customizado simula vista externa noturna urbana.

SEÇÃO 3 — CONFIGURAÇÃO TÉCNICA DO PROJETO

3.1 Versão do Unity e Porquê

Unity 6000.0.68f1 (Unity 6)

Escolhida por ser a versão mais recente da geração Unity 6, com suporte nativo ao Universal Render Pipeline (URP) 17.x — necessário para os efeitos visuais sci-fi planejados (bloom, vinheta, aberração cromática via URP Volume). O Unity 6 oferece compatibilidade total com o Meta XR SDK 68.x e OpenXR 1.16, além de melhorias no sistema de RenderTexture com URP, fundamental para a captura de frames das câmeras virtuais.

3.2 Instalação do Meta XR SDK (Passo a Passo)

Nota importante: *Este projeto utiliza exclusivamente o Meta XR SDK para toda a lógica de VR, interação e simulação. O XR Interaction Toolkit (pacote separado da Unity) NÃO é utilizado, pois causaria conflitos de componentes com o sistema de interação nativo do Meta SDK.*

1. Abrir Unity Hub → criar projeto 3D (URP Template), nome: SentinelVR
2. Acessar Window > Package Manager
3. Dropdown superior: selecionar "My Assets" (requer login na Unity Asset Store)
4. Buscar por "Meta XR All-in-One SDK"
5. Clicar em "Download" (aguardar ~500MB) → clicar em "Import"
6. Na janela de importação, manter marcados:
 - **Core SDK** — componentes base OVR (OVRCameraRig, OVRInput)
 - **Interaction SDK** — sistema de interação nativo Meta (Ray Interactor, Poke Interactor)
 - **Building Blocks** — blocos prontos para rig de câmera e mãos
 - **Spatial Audio** — OVRSpatialAudio para áudio direcional 3D nos alertas
7. Desmarcar: Sample Scenes, XR Interaction Toolkit Integration (evitar conflitos)

8. Clicar em "Import" e aguardar compilação
1. Verificar no Console: ausência de erros críticos
2. Verificar criação da pasta Assets/Meta no Project Window

3.3 Configurações de Build para Android/Meta Quest

Switch de Plataforma:

```
File > Build Settings > Android > Switch Platform
// Aguardar recompilação de shaders (10-15 min)
```

Player Settings (aba Android):

- **Package Name:** com.TIC29.SentinelVR
- **Minimum API Level:** Android 10.0 (API Level 29) — compatibilidade Quest 2/3
- **Target API Level:** Android 13.0 (API Level 33)
- **Scripting Backend:** IL2CPP (obrigatório para ARM64)
- **Target Architectures:** ARM64 apenas (desmarcar ARMv7)
- **Texture Compression:** ASTC (melhor qualidade/tamanho para GPU Adreno)
- **Graphics API:** Vulkan (primário), OpenGLES3 (fallback)

Quality Settings:

```
Edit > Project Settings > Quality > Android
VSync Count: Don't Sync
Anti Aliasing: 4x Multi Sampling
Shadow Resolution: Medium
```

3.4 Configuração do XR Plugin Management (Meta XR SDK)

3. Edit > Project Settings > XR Plugin Management
4. Selecionar aba Android (ícone do robô)
5. Marcar "Meta XR" (ou "Oculus") na lista de Plug-in Providers
6. Aguardar instalação automática de dependências
7. Expandir as configurações do Meta XR:
 - **Stereo Rendering Mode:** Multiview (melhor performance no Quest)
 - **Target Devices:** Quest 2 + Quest 3
 - **Phase Sync:** Habilitado (reduz latência de renderização)
 - **Application SpaceWarp:** Habilitado (mantém 72 FPS em cenas pesadas)
 - **Tracking Origin Mode:** Floor Level

Nota: NÃO habilitar outros providers (OpenXR genérico, Windows Mixed Reality, etc.) na mesma plataforma — causaria conflito com o Meta XR SDK.

3.5 Movimentação no PC (Editor) — Meta XR Simulator

Solução: Meta XR Simulator — ferramenta nativa do Meta XR SDK para simular o headset e controladores no Editor, sem necessidade de hardware físico.

Diferente do XR Device Simulator (pacote separado do XR Interaction Toolkit), o Meta XR Simulator é integrado ao Meta XR SDK e simula fielmente o comportamento dos controladores Meta Quest, incluindo inputs do OVRInput.

8. No Unity Editor: Meta > Simulator > Enable Meta XR Simulator
9. O painel do Simulator abre automaticamente
10. Configurar inputs do teclado/mouse conforme tabela:

Ação	Controle (Meta XR Simulator)
Mover câmera (headset)	Botão direito do mouse + WASD

Subir / Descer	Q / E
Rotacionar visão	Mouse (botão direito pressionado)
Mover controlador esquerdo	Shift + Mouse
Mover controlador direito	Ctrl + Mouse
Trigger esquerdo (OVRInput)	T
Trigger direito (OVRInput)	N
Grip direito (OVRInput)	B

SEÇÃO 4 — ASSETS E ELEMENTOS DA CENA

#	Nome	Tipo	Origem	Função
A1	Dark Metal Panel Texture	Textura PBR 2048x2048	Poly Haven — metal_plate_002 (CC0)	Revestimento de paredes e teto da sala. Normal map para profundidade.
A2	Hexagonal Control Desk	Objeto 3D (ProBuilder)	Primitivos Unity (Cylinder + Cubes)	Mesa central hexagonal de vidro translúcido onde ficam os monitores virtuais. ~800 tri.
A3	Monitor Screen Frame	Objeto 3D (FBX)	Unity Asset Store — Simple Monitor Pack (Gratuito)	6 frames em semicírculo; cada um recebe RenderTexture da câmera correspondente (~500 tri).
A4	Surveillance Cameras (x6)	Objeto 3D + Camera Component	Primitivos Unity + Unity Camera Component	Câmeras nas salas secundárias; cada uma renderiza para RenderTexture 512x512.
A5	Sci-fi LED Emissive Material	Material URP Lit	Criado no Unity (Emission habilitada)	Material azul elétrico #00BFFF, Intensity 2.0, aplicado em detalhes das paredes e rodapés.
A6	Alert Panel Texture	Textura 2D PNG 1024x1024	Canva (uso educacional)	Textura de painel de alerta com ícones; alternada entre estado normal e alerta via script.
A7	Alarm Sound Effect	Áudio WAV estéreo	Freesound.org — alarm_short_beep (CC0)	Reproduzido ao detectar anomalia. 880Hz, 0.5s, vol 60%.
A8	Ambient Hum Loop	Áudio WAV mono (loop)	Freesound.org — server_room_hum (CC0)	Som ambiente de sala de servidores. Vol 15%, loop contínuo.
A9	Procedural Sci-fi Skybox	Material Skybox	Unity Asset Store — Skybox Series Free (Gratuito)	Skybox de cidade futurista noturna visível pelas janelas virtuais da sala.
A10	Holographic Particles	Particle System	Unity Built-in Particle System	~200 partículas azuis translúcidas (#00BFFF, alpha 0.3) flutuando no ar. Atmosfera sci-fi.

SEÇÃO 5 — HIERARQUIA DE GAME OBJECTS

Scene: SentinelVR_ControlRoom

Nota: Todos os componentes de VR utilizam exclusivamente o Meta XR SDK. Não há componentes do XR Interaction Toolkit na hierarquia.

```
SentinelVR_ControlRoom
```

```

+-- [--- MANAGEMENT ---]
|   +-- EventSystem
|   |   +-- (Component: EventSystem + OVRInputModule) // Meta SDK, nao
XRUIInputModule
|   +-- OVR Manager
|   |   +-- (Component: OVRManager - tracking, refresh rate, boundary)
|   +-- AudioManager
|   |   +-- (Component: Audio Source - Ambient Hum Loop, loop: true, vol: 0.15)
|   |   +-- AlertAudioSource
|   |   +-- (Component: Audio Source + OVRSpatialAudioSource - spatialBlend:
1.0)
|   +-- AnomalySystem
|   |   +-- (Script: AnomalyDetector.cs - captura frames + WebSocket)
|   |   +-- (Script: AnomalyAlertController.cs - gerencia alertas visuais + audio
espacial)
|
+-- [--- PLAYER ---]
|   +-- OVRCameraRig // Building Block do Meta XR SDK - substitui XR Origin
|   |   +-- TrackingSpace
|   |   |   +-- CenterEyeAnchor
|   |   |   |   +-- (Component: Camera, Audio Listener, URP Camera Data)
|   |   |   +-- LeftHandAnchor
|   |   |   |   +-- OVRHand_Left
|   |   |   |   |   +-- (Component: OVRHand - rastreamento de mao)
|   |   |   |   +-- RayInteractor_Left // Meta Interaction SDK
|   |   |   |   |   +-- (Component: RayInteractor - Meta SDK, nao XR Ray Interactor)
|   |   |   |   |   +-- (Component: LineRenderer - Ray Visual, cor branca)
|   |   |   +-- RightHandAnchor
|   |   |   |   +-- OVRHand_Right
|   |   |   |   +-- RayInteractor_Right // Meta Interaction SDK
|   |   |   |   |   +-- (Component: RayInteractor - Meta SDK)
|   |   |   |   |   +-- (Component: LineRenderer - Ray Visual, cor branca)
|   |
+-- [--- ENVIRONMENT ---]
|   +-- Room_Structure
|   |   +-- Floor (Plane 12x10, Material: Dark_Metal_Panel)
|   |   +-- Ceiling (Plane 12x10, Material: Dark_Metal_Panel)
|   |   +-- Wall_North
|   |   |   +-- (Cube escalado, Material: Dark_Metal_Panel)
|   |   |   +-- LED_Strips_North (x3 Quads, Material: Scifi_LED_Blue)
|   |   +-- Wall_South
|   |   |   +-- (Cube escalado, Material: Dark_Metal_Panel)
|   |   |   +-- StatusBoard_Main
|   |   |   |   +-- (Canvas World Space - metricas do modelo de IA)
|   |   |   |   +-- (Script: AIMetricsDisplay.cs)
|   |   +-- Wall_East / Wall_West (janelas com Glass_Transparent)
|   +-- Lighting
|   |   +-- Ambient_Light (Directional, Intensity: 0.15, cor: #0A0A2E)
|   |   +-- PointLight_Station01..03 (Intensity: 1.2, Range: 3m)
|   |   +-- LED_Ambient_Fill (Point, Intensity: 0.4, cor: #00BFFF)
|   +-- Equipment_Decorative (ServerRack_Left/Right, Cable_Conduits)
|   +-- Atmosphere
|   |   +-- Skybox_Controller (Material: Scifi_Night_City)
|   |   +-- Holographic_Particles (200 particulas, #00BFFF, alpha: 0.3)
|
+-- [--- MAIN_CONTENT - CONTROL_CENTER ---]
|   +-- HexDesk_Central
|   |   +-- Desk_Surface (Cylinder achatado, Glass_Translucent)
|   |   +-- Desk_Base (Cylinder, Dark_Metal_Panel)
|   +-- Monitor_Array (semicirculo - 6 monitores, abrangendo 180 graus)
|   |   +-- Monitor_01
|   |   |   +-- Screen_Frame (FBX, Monitor_Frame_Dark)
|   |   |   +-- Screen_Display (Quad, Material com RenderTexture_Cam01)
|   |   |   +-- AlertBorder_01 (Quad borda, Alert_Border_Red)
|   |   |   +-- AlertAudio_01 (OVRSpatialAudioSource - som no espaco 3D da camara)
|   |   |   +-- (Script: MonitorController.cs - indice: 0)
|   |   +-- Monitor_02..06 (mesma estrutura, indices 1-5)
|   |   +-- AlertPanel_Master (Canvas World Space, central, grande)
|   |   |   +-- Text_AlertStatus / Text_AnomalyScore (TextMeshPro)
|   |   |   +-- Icon_AlertSymbol (Image)

```

```

|   +-- TeleportationHub
|   +-- TeleportArea_MainStation (Component: TeleportationArea - Meta SDK)
|   +-- TeleportArea_SecondaryView (Component: TeleportationArea - Meta SDK)
|
+-- [--- SURVEILLANCE_ROOMS (ocultas a +50m no eixo X) ---]
|   +-- Room_A
|   |   +-- Geometry_RoomA (paredes, piso)
|   |   +-- Camera_01
|   |   |   +-- (Component: Camera, Target Texture: RenderTexture_Cam01)
|   |   |   +-- (Script: CameraCapture.cs - captura frames para Python)
|   |   +-- MovingObject_A1 (Script: PatrolMovement.cs)
|   |   +-- AnomalyObject_A (Script: AnomalyTrigger.cs - ativado randomicamente)
|   +-- Room_B..F (mesma estrutura, camaras 02-06)
|
+-- [--- INTERACTABLES ---]
+-- DismissButton_01..06
|   +-- (Sphere primitivo, ao lado de cada monitor)
|   +-- (Component: PointableUnityEventWrapper - Meta Interaction SDK)
|   +-- (Script: DismissAlert.cs)
|   +-- Tag: Interagivel
+-- SystemToggle_Button
|   +-- (Cube primitivo, mesa central)
|   +-- (Component: PointableUnityEventWrapper - Meta Interaction SDK)
|   +-- (Script: ToggleAISystem.cs)

```

Lógica de Ocultação das Salas: As Surveillance_Rooms são posicionadas a +50m no eixo X, fora do alcance visual do player. Apenas as câmeras virtuais as capturam via RenderTexture, gerando o efeito de monitoramento real sem que o operador possa acessá-las.

SEÇÃO 6 — SISTEMA DE INTERAÇÃO

6.1 Descrição da Interação Principal

O operador está imerso na sala de controle com 6 monitores em semicírculo. Ao apontar o raio do controlador (Meta RayInteractor) para um DismissButton abaixo de cada monitor, o botão emite brilho verde via evento OnHoverEntered do Meta Interaction SDK. Ao pressionar o Trigger (OVRInput.Get(OVRInput.Button.PrimaryIndexTrigger)), o alerta é dispensado: a borda vermelha apaga, o painel central atualiza e o áudio espacial do monitor correspondente é silenciado. O botão SystemToggle na mesa pausa/retoma o pipeline de IA com feedback visual no StatusBoard.

A experiência VR é essencial ao fluxo de interação: o operador usa a proprioception (senso de posição do corpo no espaço) para identificar de qual câmera veio o alerta pelo som 3D antes de olhar — algo que uma interface 2D não consegue oferecer.

6.2 Fluxo de Interação

Fluxo A — Alerta Emitido pelo Modelo de IA:

11. CameraCapture.cs captura frame da Camera_01 a cada 0.5s via RenderTexture e converte para PNG bytes.
12. Frame é enviado via WebSocket para servidor Python (porta 8765) como mensagem binária.
13. Python recebe → decodifica → extrai features via ResNet50 → Autoencoder calcula MSE.
14. Se MSE > 0.045: Python responde JSON {"camera": 0, "score": 0.087, "is_anomaly": true}.
15. AnomalyDetector.cs recebe JSON → dispara OnAnomalyDetected(cameraIndex, score).
16. AnomalyAlertController.cs: ativa borda vermelha piscante no Monitor_01.

17. OVRSpatialAudioSource do AlertAudio_01 reproduz beep no espaço 3D da câmera — operador ouve de onde vem.
18. AlertPanel_Master exibe: "ANOMALIA — CAM 01 | Score: 0.087".

Fluxo B — Operador Dispensa o Alerta:

19. Operador ouve áudio espacial, vira o corpo na direção da câmera (uso natural da VR).
20. Aponta RayInteractor para DismissButton_01 →
PointableUnityEventWrapper.WhenPointerEventRaised.
21. OnHoverEntered → botão aumenta escala 10% e emite brilho verde.
22. Pressiona Trigger → OVRInput.GetDown() → OnSelectEntered.
23. DismissAlert.cs chama MonitorController.DismissAlert(): borda desativa, áudio silencia.
24. AlertPanel_Master atualiza: "ALERTA CAM 01 DISPENSADO". Log com timestamp no StatusBoard.

6.3 Scripts C# — Principais

AnomalyDetector.cs — Captura de Frames e WebSocket

```
using UnityEngine;
using UnityEngine.Events;
using NativeWebSocket;
using System;
using System.Collections;

/// <summary>
/// Captura frames das cameras de vigilancia e envia para o servidor Python.
/// Utiliza Meta XR SDK: OVRInput para interacoes, NativeWebSocket para comunicacao.
/// SentinelVR - Residencia TIC 29
/// </summary>
public class AnomalyDetector : MonoBehaviour
{
    [Header("Cameras de Vigilancia")]
    [SerializeField] private Camera[] surveillanceCameras;
    [SerializeField] private RenderTexture[] cameraRenderTextures;
    [SerializeField] private float captureInterval = 0.5f;

    [Header("WebSocket")]
    [SerializeField] private string serverUrl = "ws://localhost:8765";

    [Header("Eventos")]
    public UnityEvent<int, float> OnAnomalyDetected;
    public UnityEvent<int> OnAnomalyCleared;

    private WebSocket websocket;
    private Texture2D captureBuffer;
    private bool isConnected = false;
    private int currentCameraIndex = 0;

    private async void Start()
    {
        captureBuffer = new Texture2D(512, 512, TextureFormat.RGB24, false);
        websocket = new WebSocket(serverUrl);
        websocket.OnOpen += () => { isConnected = true; };
        websocket.OnMessage += (bytes) => ProcessAnomalyResponse(
            System.Text.Encoding.UTF8.GetString(bytes));
        websocket.OnClose += (code) => { isConnected = false; };
        await websocket.Connect();
        InvokeRepeating(nameof(CaptureNextCamera), 2f, captureInterval);
    }

    private void Update()
    {
        // Meta XR SDK: despachar mensagens WebSocket no loop principal
        #if !UNITY_WEBGL || UNITY_EDITOR
```



```

        websocket?.DispatchMessageQueue();
    #endif
}

private void CaptureNextCamera()
{
    if (!isConnected || surveillanceCameras.Length == 0) return;
    currentCameraIndex = (currentCameraIndex + 1) % surveillanceCameras.Length;
    StartCoroutine(CaptureAndSend(currentCameraIndex));
}

private IEnumerator CaptureAndSend(int idx)
{
    yield return new WaitForEndOfFrame();
    RenderTexture rt = cameraRenderTextures[idx];
    RenderTexture prev = RenderTexture.active;
    RenderTexture.active = rt;
    captureBuffer.ReadPixels(new Rect(0, 0, rt.width, rt.height), 0, 0);
    captureBuffer.Apply();
    RenderTexture.active = prev;
    byte[] frameBytes = captureBuffer.EncodeToPNG();
    byte[] payload = new byte[4 + frameBytes.Length];
    Buffer.BlockCopy(BitConverter.GetBytes(idx), 0, payload, 0, 4);
    Buffer.BlockCopy(frameBytes, 0, payload, 4, frameBytes.Length);
    if (websocket.State == WebSocketState.Open)
        await websocket.Send(payload);
}

private void ProcessAnomalyResponse(string json)
{
    AnomalyResult r = JsonUtility.FromJson<AnomalyResult>(json);
    if (r.is_anomaly) OnAnomalyDetected?.Invoke(r.camera, r.score);
    else OnAnomalyCleared?.Invoke(r.camera);
}

private async void OnApplicationQuit()
{
    if (websocket != null) await websocket.Close();
}

[Serializable] private class AnomalyResult
{
    public int camera; public float score; public bool is_anomaly;
}

```

MonitorController.cs — Controle Visual + Áudio Espacial (Meta SDK)

```

using UnityEngine;
using TMPro;
using System.Collections;

/// <summary>
/// Controla estado visual + audio espacial de cada monitor.
/// AlertAudioSource usa OVRSpatialAudioSource do Meta XR SDK.
/// </summary>
public class MonitorController : MonoBehaviour
{
    [Header("Componentes Visuais")]
    [SerializeField] private GameObject alertBorder;
    [SerializeField] private TextMeshProUGUI scoreLabel;

    [Header("Audio Espacial (Meta XR SDK - OVRSpatialAudioSource)")]
    [SerializeField] private AudioSource alertAudioSource; // com
    OVRSpatialAudioSource

    [Header("Configuracoes")]
    [SerializeField] private int monitorIndex;
    [SerializeField] private float blinkInterval = 0.4f;

    private bool isAlerting = false;
    private Coroutine blinkCoroutine;

```



```

private void Awake() { alertBorder.SetActive(false); }

public void TriggerAlert(float anomalyScore)
{
    if (isAlerting) return;
    isAlerting = true;
    alertBorder.SetActive(true);
    scoreLabel.text = $"SCORE: {anomalyScore:F3}";
    scoreLabel.gameObject.SetActive(true);
    // Audio espacial: som emitido na posicao 3D deste monitor no espaco VR
    alertAudioSource.Play();
    blinkCoroutine = StartCoroutine(BlinkBorder());
}

public void DismissAlert()
{
    if (!isAlerting) return;
    isAlerting = false;
    if (blinkCoroutine != null) StopCoroutine(blinkCoroutine);
    alertBorder.SetActive(false);
    scoreLabel.gameObject.SetActive(false);
    alertAudioSource.Stop(); // silencia audio espacial deste monitor
}

private IEnumerator BlinkBorder()
{
    {
        bool visible = true;
        while (isAlerting)
        {
            alertBorder.SetActive(visible);
            visible = !visible;
            yield return new WaitForSeconds(blinkInterval);
        }
    }
}
}

```

DismissAlert.cs — Interação com Meta Interaction SDK

```

using UnityEngine;
using Oculus.Interaction; // namespace do Meta Interaction SDK

/// <summary>
/// Dispensa o alerta do monitor via Meta Interaction SDK.
/// PointableUnityEventWrapper chama OnSelect quando o usuario pressiona o trigger
/// enquanto o RayInteractor (Meta SDK) aponta para este objeto.
/// NAO usa XR Simple Interactable (XR Interaction Toolkit).
/// </summary>
[RequireComponent(typeof(PointableUnityEventWrapper))]
public class DismissAlert : MonoBehaviour
{
    [SerializeField] private MonitorController targetMonitor;

    private PointableUnityEventWrapper _interactable;

    private void Awake()
    {
        _interactable = GetComponent<PointableUnityEventWrapper>();
        _interactable.WhenPointerEventRaised.AddListener(OnPointerEvent);
    }

    private void OnPointerEvent(PointerArgs args)
    {
        if (args.PointerEvent == PointerEvent.Select)
            targetMonitor?.DismissAlert();
    }
}

```

anomaly_server.py — Servidor Python (sem alterações de SDK)

```

"""
SentinelVR - Servidor de Detecção de Anomalias
Recebe frames PNG do Unity via WebSocket,
extrai features com ResNet50 e detecta anomalias via Autoencoder.
"""
import asyncio, websockets, numpy as np, json, struct
from io import BytesIO
from PIL import Image
import torch, torch.nn as nn
import torchvision.transforms as transforms
import torchvision.models as models

HOST, PORT = 'localhost', 8765
ANOMALY_THRESHOLD = 0.045

class AnomalyAutoencoder(nn.Module):
    def __init__(self, input_dim=2048, latent_dim=64):
        super().__init__()
        self.encoder = nn.Sequential(
            nn.Linear(input_dim, 512), nn.ReLU(), nn.Dropout(0.2),
            nn.Linear(512, 128), nn.ReLU(), nn.Linear(128, latent_dim), nn.ReLU())
        self.decoder = nn.Sequential(
            nn.Linear(latent_dim, 128), nn.ReLU(),
            nn.Linear(128, 512), nn.ReLU(), nn.Dropout(0.2),
            nn.Linear(512, input_dim), nn.Sigmoid())
        def forward(self, x): return self.decoder(self.encoder(x))
        def reconstruction_error(self, x):
            with torch.no_grad(): return nn.functional.mse_loss(self.forward(x),
x).item()

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
resnet = models.resnet50(weights=models.ResNet50_Weights.DEFAULT)
feature_extractor = nn.Sequential(*list(resnet.children())[:-1]).eval().to(device)
autoencoder = AnomalyAutoencoder().to(device)
preprocess = transforms.Compose([
    transforms.Resize((224, 224)), transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])])

async def handle_frame(ws):
    async for msg in ws:
        if isinstance(msg, bytes) and len(msg) > 4:
            cam_idx = struct.unpack('<i', msg[:4])[0]
            img = Image.open(BytesIO(msg[4:])).convert('RGB')
            t = preprocess(img).unsqueeze(0).to(device)
            with torch.no_grad(): feats = feature_extractor(t).view(1, -1)
            score = autoencoder.reconstruction_error(feats)
            await ws.send(json.dumps({
                'camera': cam_idx, 'score': round(score, 4),
                'is_anomaly': score > ANOMALY_THRESHOLD}))

async def main():
    async with websockets.serve(handle_frame, HOST, PORT): await asyncio.Future()

if __name__ == '__main__': asyncio.run(main())

```

SEÇÃO 7 — PLANEJAMENTO DO REPOSITÓRIO GITHUB

7.1 Nome do Repositório

SentinelVR-TIC29

7.2 Estrutura de Pastas

```

SentinelVR-TIC29/
|

```

```

+-- Assets/
|   +-- Scenes/
|   |   +-- SentinelVR_ControlRoom.unity
|   +-- Scripts/
|   |   +-- AI/           (AnomalyDetector.cs, AnomalyAlertController.cs)
|   |   +-- Monitors/    (MonitorController.cs, DismissAlert.cs)
|   |   +-- Surveillance/ (CameraCapture.cs, PatrolMovement.cs)
|   |   +-- UI/          (AIMetricsDisplay.cs, ToggleAISystem.cs)
|   +-- Materials/       (Dark_Metal_Panel.mat, Scifi_LED_Blue.mat, ...)
|   +-- RenderTextures/  (RT_Camera_01..06.renderTexture)
|   +-- Audio/           (alarm_beep.wav, server_room_hum.wav)
|   +-- Textures/        (metal_plate_002_diffuse.png, ...)
+-- PythonServer/
|   +-- anomaly_server.py
|   +-- train_autoencoder.py
|   +-- requirements.txt
|   +-- sentinel_autoencoder.pth
+-- ProjectSettings/
|   +-- ProjectSettings.asset
|   +-- QualitySettings.asset
|   +-- XRSettings.asset
+-- Packages/
|   +-- manifest.json      // Meta XR SDK sem XR Interaction Toolkit
|   +-- packages-lock.json
+-- .gitignore             // ignora /Library, /Temp, /Logs, *.pth
+-- README.md

```

SEÇÃO 8 — PLANO DE EXECUÇÃO PASSO A PASSO

25. **Etapa 1 — Setup do Projeto Unity (URP):** Criar projeto 3D URP no Unity 6000.0.68f1. Configurar pastas base. Definir Package Name e identidade em Player Settings.
26. **Etapa 2 — Meta XR SDK Exclusivo:** Importar Meta XR All-in-One SDK via Package Manager (Core SDK + Interaction SDK + Building Blocks + Spatial Audio). Configurar XR Plugin Management com Meta XR provider em modo Multiview. NÃO instalar XR Interaction Toolkit.
27. **Etapa 3 — OVRCameraRig e Mãos do Player:** Usar Meta Building Blocks para adicionar OVRCameraRig com OVRHand e RayInteractor (Meta SDK) em ambas as mãos. Configurar Meta XR Simulator para testes no Editor sem headset físico.
28. **Etapa 4 — Construção da Sala de Controle:** Modelar ambiente com primitivos Unity. Mesa central hexagonal com ProBuilder. Configurar iluminação ambiente azul escuro + point lights. Posicionar 6 monitores em semicírculo de 180° ao redor do ponto de presença do operador.
29. **Etapa 5 — RenderTextures e Câmeras de Vigilância:** Criar 6 RenderTexture assets (512×512, RGB24). Posicionar câmeras nas Surveillance Rooms (+50m eixo X). Criar materiais para monitores usando RenderTextures como albedo. Configurar OVRSpatialAudioSource em cada AlertAudio_01..06.
30. **Etapa 6 — Servidor Python:** Implementar anomaly_server.py com WebSocket assíncrono, ResNet50 + Autoencoder. Treinar com frames normais capturados (train_autoencoder.py). Calibrar threshold = 0.045.
31. **Etapa 7 — Integração Unity ↔ Python via WebSocket:** Importar NativeWebSocket. Implementar AnomalyDetector.cs (captura → WebSocket) e MonitorController.cs (visual + áudio espacial). Adicionar DismissButton com PointableUnityEventWrapper (Meta Interaction SDK) e DismissAlert.cs.
32. **Etapa 8 — Testes e Build:** Testar fluxo completo com Meta XR Simulator + servidor Python. Verificar FPS > 72. Confirmar que áudio espacial direciona atenção corretamente. Build Android APK para Meta Quest.

SEÇÃO 9 — REFLEXÃO FINAL

9.1 Aprendizado

O projeto SentinelVR consolidou dois aprendizados complementares. O primeiro é arquitetural: a separação de responsabilidades entre ambiente de execução (Unity + Meta XR SDK) e módulo de inferência (Python + PyTorch) via WebSocket — o padrão inference-as-a-service presente em sistemas de segurança comerciais reais.

O segundo, e mais relevante para a disciplina, é conceitual: VR não é apenas uma interface alternativa — é um meio com propriedades únicas (espacialidade, presença, áudio direcional, memória motora) que devem ser exploradas para que o projeto justifique a escolha do metaverso. O SentinelVR demonstra isso ao tornar o áudio espacial e a disposição 360° dos monitores partes funcionais da solução, não elementos decorativos.

A distinção entre o Meta XR SDK (sistema completo de VR da Meta com OVRCameraRig, Meta Interaction SDK, OVRSpatialAudio) e o XR Interaction Toolkit (pacote genérico da Unity para múltiplas plataformas XR) foi essencial para manter a coerência técnica: para um projeto voltado exclusivamente ao Meta Quest, o Meta XR SDK oferece integração mais profunda e menor overhead de abstrações desnecessárias.

9.2 Dificuldades Previstas

Latência do Pipeline de IA: A cadeia completa (captura PNG → WebSocket → feature extraction → Autoencoder → resposta) deve ficar abaixo de 200ms. Estratégias: async no servidor, reduzir captura para 224×224 direto, processar apenas a câmera com maior atividade recente.

Calibração do Threshold de Anomalia: Threshold baixo demais gera falsos positivos constantes; alto demais deixa anomalias passarem. Plano: capturar ao menos 2000 frames normais por câmera antes de treinar.

Performance de RenderTextures e Áudio Espacial: Seis câmeras + seis OVRSpatialAudioSources simultâneos a 72 FPS no Quest podem sobrecarregar o hardware. Solução: alternar câmeras em Coroutine (uma por frame) e usar culling de distância nos AudioSources.

9.3 Melhorias Futuras

Eye Tracking para Atenção Adaptativa: Com Meta Quest Pro (eye tracking nativo no Meta XR SDK via OVREyeGaze), o sistema pode detectar para qual monitor o operador está olhando e priorizar o processamento de IA naquelas câmeras, reduzindo carga computacional.

LSTM para Anomalias Temporais: Substituir o Autoencoder estático por LSTM Autoencoder para detectar anomalias comportamentais ao longo do tempo (trajetória incomum), não apenas em frames isolados.

Dashboard Analytics Externo: Exportar logs de anomalias para Streamlit ou Dash, com série temporal de scores por câmera e mapa de calor de frequência de anomalias por local.